Python Mini-Projects

Guided, then unguided, then one major project

Topic 1: Variables, Types, Input/Output, Operators
Topic 2: Conditions and Loops
Topic 3: Lists and Dictionaries
Topic 4: Functions
Major Project: School Records Manager

How This Pack Works

This pack is about building, not answering quiz questions. In each topic you build small programs, with less and less help each time. By the end you will have built a complete program of your own.

The three levels in every topic

Level	What you get	What it builds
Guided	Numbered steps and a sample output. Hints are there if you get stuck.	Confidence with the syntax and following a plan.
Unguided	A short brief only. No steps, no hints, no starter code.	Planning, breaking a problem down, and writing code on your own.
Major project	A full brief that combines Topics 1 to 4, built in phases.	Designing and connecting ideas like a real developer.

How to work through it

1. Do the guided projects of a topic first. Try each one without the hints, then open the hints if you need them.
2. Move on to the unguided project. Spend a good 10 minutes on your own before you look anything up.
3. Answer the "Explain it" questions at the end of the topic, out loud or in writing.
4. Go to the next topic when your unguided project works.
5. Finish with the Major Project.

Good habits

- Run your code often. After every 2 or 3 lines, not only at the end.
- Read the error message from the bottom up. The last line tells you what went wrong.
- Using Google or notes is fine, but be able to explain every line you write.
- A program that works but looks messy is better than a perfect plan with no code.
- Getting stuck is part of programming. It means you are learning something.

Time guide

Part	Suggested time
Each guided project	15 to 25 minutes
Each unguided project	30 to 45 minutes
Major project	3 to 5 hours, over several sessions

TOPIC 1

Variables, Data Types, Input/Output and Operators

Concepts covered
variables, str / int / float, input(), print(), f-strings, type conversion, arithmetic operators, round()

GUIDED 1.1 Student Profile Card

Goal: variables, input(), type conversion, f-strings.

1. Ask the user for their **name**, **age**, **city** and **favourite subject**. Store each in a variable.
2. Convert the age to an integer.
3. Calculate the **year they were born**, using the current year.
4. Calculate **how old they will be in 5 years**.
5. Print a profile card like the sample below.

```
========================
Name: Amina
Age: 17 (born around 2009)
City: Nairobi
Favourite subject: Maths
In 5 years you will be 22
========================
```

Hints (try without them first)
input() always returns text (a string). int() turns text into a whole number. An f-string looks like f"Name: {name}".

GUIDED 1.2 Shop Receipt Calculator

Goal: float(), arithmetic operators, round().

1. Ask for the **item name**, **price per item** and **quantity**.
2. Calculate the **subtotal** (price x quantity).
3. Add **16% VAT**.
4. Print a receipt with every line labelled and amounts rounded to 2 decimal places.
5. *Stretch:* apply a discount of 500 if the total is above 5,000 (introduces if).

Think about it: Why do you use float() for the price and int() for the quantity? Why does `"5" * 3` give `"555"` and not 15?

UNGUIDED 1.3 School Fee Balance Tracker

Brief only. No steps, no hints, no starter code.

Brief
Build a program for a school bursar. It asks for a student's name, the total term fee and the amount paid so far. It shows the **balance remaining**, the **percentage of the fee paid**, and a message such as "Hello Amina, your balance is KES 12,000." It also shows how much the student would owe **per month** if the balance were split over 3 months.

Must include: at least 4 variables, input() with correct conversion, arithmetic operators, f-string output, round() for decimals.

Extension: a currency toggle (KES or USD), or a neat multi-line statement.

Explain it (be ready to answer these)
1. Why does input() return a string? 2. What happens if you forget int() before doing maths? 3. What is the difference between / and //?

TOPIC 2

Conditions and Loops

Concepts covered
if / elif / else, comparison operators, and / or / not, while loops, for loops, range(), break, continue

GUIDED 2.1 Exam Grade Calculator

Goal: if / elif / else and comparison operators.

1. Ask for the student's name and their marks in **3 subjects** (each out of 100).
2. Calculate the **average**.
3. Give a grade: 80 and above = A, 70 to 79 = B, 60 to 69 = C, 50 to 59 = D, below 50 = E.
4. Print the average and the grade.
5. Print "PASS" if the average is 50 or more, otherwise "FAIL".
6. *Stretch:* if any mark is above 100 or below 0, print an error message and do not calculate.

```
Name: Amina
Maths: 78
English: 65
Science: 82
Average: 75.0
Grade: B
Result: PASS
```

Hints (try without them first)
Check the highest grade first and work down. Use >= for "and above". Use "and" to check a range, e.g. mark >= 0 and mark <= 100.

GUIDED 2.2 Number Guessing Game

Goal: while loop, break, counters, random.

1. At the top, write `import random`. Then pick a secret number: `secret = random.randint(1, 20)`.
2. Create a counter **attempts** that starts at 0.
3. Use a **while** loop to keep asking the user to guess.
4. After each guess, add 1 to attempts and print "Too high", "Too low" or "Correct!".
5. Stop the loop when the guess is correct (use break).
6. At the end, print how many attempts it took.
7. *Stretch:* limit the player to 5 attempts and reveal the number if they lose.

GUIDED 2.3 Multiplication Table Printer

Goal: for loop and range().

1. Ask the user for a number.
2. Use a **for** loop with range() to print its table from 1 to 12, one line each, e.g. 7 x 3 = 21.
3. *Stretch:* print the table for every number from 1 to 5 using a loop inside a loop.

UNGUIDED 2.4 ATM Withdrawal Simulator

Brief only. No steps, no hints.

Brief
Build a simple ATM. The account starts with a balance of 20,000. The user has **3 attempts** to enter the correct PIN (choose one yourself). After a correct PIN, the program repeatedly asks how much to withdraw. It must refuse an amount that is more than the balance, or zero or negative, and it must show the new balance after every successful withdrawal. The user types 0 to exit. After 3 wrong PINs, the card is blocked and the program ends.

Must include: a while loop, if / elif / else, a counter for PIN attempts, and a clear exit condition.

Extension: a daily limit of 15,000, and a count of how many withdrawals were made.

Explain it (be ready to answer these)
1. What is the difference between break and continue? 2. What causes an infinite loop, and how do you stop one? 3. When would you pick for instead of while?

TOPIC 3

Lists and Dictionaries

Concepts covered
creating lists, indexing, append / remove / sort, len / sum / max / min, looping over lists, dictionaries (key and value), looping over dictionaries, list of dictionaries

GUIDED 3.1 Class Register

Goal: lists, append, loops over lists, len().

1. Start with an empty list called **students**.
2. Use a loop to ask for **5 student names** and add each to the list with append().
3. Print the full list.
4. Print the **first** and the **last** student, and the **total** number of students.
5. Sort the list alphabetically and print it again, numbered 1 to 5.
6. Ask the user for a name and say whether that student is in the class (use in).
7. *Stretch:* let the user remove a student by name.

```
1. Amina
2. Bilal
3. Hodan
4. Khadija
5. Yusuf
```

GUIDED 3.2 Marks Analyser

Goal: list of numbers, sum(), max(), min(), averages.

1. Create a list of marks: `marks = [45, 78, 62, 90, 55, 83, 71]`.
2. Print the **highest**, **lowest**, **total** and **average** mark.
3. Use a loop to count how many marks are **50 or above** (the passes).
4. Build a **new list** that holds only the passing marks and print it.
5. *Stretch:* do all of this again, but with marks typed in by the user.

GUIDED 3.3 Shopping Basket (Dictionary)

Goal: dictionaries, key and value, looping with .items().

1. Create a dictionary of prices: `prices = {"rice": 180, "milk": 60, "sugar": 150, "bread": 55}`.
2. Ask the user for an item name. If it is in the dictionary, print its price. If not, print "Item not found".
3. Create an empty dictionary **basket**. Let the user keep adding items and quantities until they type "done".
4. Loop through the basket with .items() and print each item, quantity and line total (price x quantity).
5. Print the **grand total**.

Hints (try without them first)
prices["rice"] gives 180. Use "rice" in prices to check a key. Use for item, qty in basket.items(): to loop.

UNGUIDED 3.4 Student Records Analyser

Brief only. No steps, no hints.

Brief
Store at least **5 students** as a **list of dictionaries**. Each student has a name, a class and a list of three marks. The program prints each student's average, then shows the **top student**, the **class average**, and a list of students **below 50 on average** who need extra help. Finally, let the user type a name to look up one student's record.

Must include: a list of dictionaries, loops, sum() and len(), and a search by name.

Extension: sort the students from highest to lowest average.

Explain it (be ready to answer these)
1. When would you use a dictionary instead of a list? 2. What happens if you ask for an index that is not in the list? 3. What is the difference between append() and extend()?

TOPIC 4

Functions

Concepts covered
def, parameters, return values, default values, calling functions, functions that call other functions, scope

GUIDED 4.1 Reusable Calculators

Goal: def, parameters, return, print vs return.

1. Write a function `add(a, b)` that **returns** the sum.
2. Write `area_of_rectangle(length, width)` that returns the area.
3. Write `celsius_to_fahrenheit(c)` that returns F = C x 9/5 + 32.
4. Write `is_even(n)` that returns True or False.
5. Call each function at least twice with different values and print the results.
6. Write one more function that **prints** instead of returning, and explain the difference in a comment.

GUIDED 4.2 Refactor the Receipt

Goal: turn old code into functions. This is how real developers work.

Take your Shop Receipt Calculator from Topic 1 and restructure it:

1. Write `calculate_subtotal(price, quantity)`.
2. Write `add_vat(amount, rate=0.16)`. Notice the **default value** for rate.
3. Write `print_receipt(item, quantity, price)`, which calls the other two functions.
4. Write a `main()` function that asks for input and calls print_receipt().
5. Call main() at the bottom of the file. The output must be the same as before.

Think about it: Why is the code easier to test, read and reuse as functions?

UNGUIDED 4.3 Mini Bank Toolkit

Brief only. No steps, no hints.

Brief
Write a small set of functions for a bank: `deposit(balance, amount)`, `withdraw(balance, amount)` (it must not allow a negative balance), `calculate_interest(balance, rate, years)` and `statement(name, balance)`, which prints a neat summary. Then write a short program that uses all four in a realistic sequence: open an account, make two deposits, one withdrawal, add interest and print the statement.

Must include: 4 functions with parameters and return values, at least one default parameter, and at least one function calling another.

Extension: keep a **list of transactions** and print it in the statement.

Explain it (be ready to answer these)
1. What is the difference between a parameter and an argument? 2. What does a function return if it has no return statement? 3. Why is a variable created inside a function not visible outside it?

MAJOR PROJECT

School Records Manager

A menu-driven console program for a school office. It brings together everything from Topics 1 to 4: input and output, conditions and loops, lists and dictionaries, and functions.

The brief
A small school wants a simple program to manage its students. The office clerk runs the program and sees a **menu**. The clerk can add students, record marks, record fee payments, view a report and search for a student. The program keeps running until the clerk chooses to exit. You decide the design, but the program must be organised into **functions**.

Sample menu

```
===== SCHOOL RECORDS MANAGER =====
1. Add a new student
2. Record marks for a student
3. Record a fee payment
4. View all students
5. Search for a student
6. Class report
7. Exit
Choose an option (1-7):
```

Build it in phases

Phase	What to build	Topics used
1. Data	Store students as a list of dictionaries. Each has: id, name, class, marks (a list), fee_total, fee_paid.	3
2. Menu	Show the menu in a while loop. Run the chosen option. Reject invalid choices. Exit on 7.	1, 2
3. Add / Search	Option 1 adds a student with a unique id. Option 5 finds a student by id or name and prints their record.	2, 3, 4
4. Marks / Fees	Option 2 adds marks (0 to 100 only). Option 3 records a payment and refuses to overpay. Show the new balance.	1, 2, 4
5. Reports	Option 4 lists all students neatly. Option 6 shows the class average, top student, number of passes and the total fees still owed.	2, 3, 4
6. Polish	Handle empty lists, bad input and duplicate ids without crashing. Add comments and clear messages.	all

Rules and requirements

- At least **8 functions**. No single function longer than about 20 lines.
- A `main()` function that runs the menu loop.
- Input must be validated. Typing letters where a number is expected must not crash the program.
- Use f-strings and round() for tidy output.
- Meaningful variable and function names. A comment above each function explaining what it does.
- Work in phases. Run and test after each phase before starting the next.

Stretch goals

- Save the students to a file (for example a .json file) when exiting and load it on start-up.
- Edit and delete a student.
- Grade every student (A to E) and print a ranked list from top to bottom.
- Fee reminder list: students whose balance is more than 50% of the total.
- A simple login (username and password) before the menu appears.

Test data to try

Test	Expected behaviour
Type "abc" at the menu	Message "Please enter a number", the menu appears again
Record a mark of 105	Refused with a clear message
Pay more than the fee balance	Refused, balance unchanged
Search for a student who does not exist	Message "Student not found", no crash
View report with no students	Message "No students yet", no crash

Self-check before you finish

Tick each line honestly. Anything you cannot tick is your next thing to practise.

Check	Done
Every menu option works, and Exit closes the program properly.	
I used functions, and each function does one clear job.	
Students are stored as a list of dictionaries.	
Wrong input (letters, negative numbers, empty lists) never crashes the program.	
My output is neat, labelled and rounded where needed.	
My variable and function names make sense, and each function has a comment.	
I tested after every phase, not only at the end.	
I can explain every part of my code in my own words.	